Universidad Autónoma de Madrid

Departamento de Informática

Proyecto de Sistemas Informáticos
Práctica - 4

Roberto Marabini

Registro de Cambios

Versión[1]	**Fecha**	**Autor**	**Descripción**
1.0	12.07.2025	RM	Primera versión.
1.1	12.08.2025	RM	Revisión general del texto.
1.2	20.09.2025	RM	2ª Revisión general del texto.
1.3	14.04.2026	RM	Algunas puntualizaciones.

[1]La asignación de versiones se realizan mediante 2 números $X.Y$. Cambios en Y indican aclaraciones, descripciones más detalladas de algún punto o traducciones. Cambios en X indican modificaciones más profundas que o bien varían el material suministrado o el contenido de la práctica.

Contents

1 Objetivos **3**

2 Implementación del Frontend **3**

2.1 Página "Songs" 5

3 Despliegue de la aplicación **10**

4 Devtools **10**

5 Test **10**

6 Resumen del trabajo a entregar **11**

7 Criterios de evaluación **11**

A Capturas de pantalla **13**

1 Objetivos

El objetivo de esta práctica es completar la implementación del proyecto *songproject* desarrollando la interfaz gráfica que utilizarán los usuarios para completar las letras de las canciones reproducidas.

2 Implementación del Frontend

En la práctica anterior se utilizó *Django* para construir una API REST. En esta nueva etapa, se empleará *Vue.js* para desarrollar la interfaz gráfica que consumirá dicha API y al que accederán los usuarios del sistema.

La implementación de la interfaz gráfica debe seguir el paradigma de *Single Page Application* (SPA), utilizando el framework *Vue.js*. Esto implica que la navegación entre las distintas páginas de la aplicación debe gestionarse completamente desde el cliente, sin conectarse al navegador excepto para cargar datos.

El primer paso será configurar un *router* que defina las URLs de la aplicación. Cada ruta debe estar asociada a una **vista** principal, la cual actuará como contenedor de los distintos **componentes** que conforman la funcionalidad de esa página. Por ejemplo, la vista correspondiente a la página de inicio puede incluir componentes como un buscador y un listado de las canciones más populares.

Listado 1: Estructura de una aplicación *Vue.js* SPA que usa un "router" para cargar vistas las cuales incluyen componentes reusables.

```
src/
    components/
        SongList.vue
        SongSearch.vue3
        ...
    views/
        HomeView.vue
        Login.vue
        ...
```

```
    router/
        index.js
    App.vue
    main.js
```

Este enfoque modular permite una mejor organización del código, facilita la reutilización de componentes y mejora la mantenibilidad del proyecto. Cada vista debe estar ubicada en el directorio `views/`, mientras que los componentes reutilizables deben organizarse en el directorio `components/`.

A continuación, se describen los puntos de acceso de la aplicación web: `home_page`, `login`, `logout`, `songs` y `faq`.

- `home_page` (véase figura 1). URL: `http://host:port/`. Requisitos: esta página será la puerta de entrada a la aplicación y debe mostrar: (1) una descripción del propósito de la aplicación, (2) un listado con las tres canciones más populares (según el valor de `number_times_played`), (3) un formulario que permita buscar canciones por título y (4) un botón que al ser pulsado muestre al usuario una canción elegida al azar. Tanto las canciones más populares como las devueltas por "search" debén ser hipervinculos que permitan acceder a la página en la que se reproduzca la canción seleccionada.

- `login` (véase figura 2). URL: `http://host:port/log-in`. Funcionalidad: validar el nombre de usuario y su contraseña. Requisitos: la API devuelve un token que debe almacenarse como una variable de sesión utilizando un "store" de `pinia`. En caso de autenticación exitosa, el usuario debe ser redirigido a la página "home" (`/`). Nota: se permite el uso ocasional de la plataforma por parte de usuarios no registrados. La diferencia principal entre usuarios registrados y anónimos es que, en el primer caso, se almacena información adicional mediante objetos del tipo `SongUser`. Se puede acceder a esta página mediante un link en el menu de la aplicación.

- `logout` (véase figura 3). URL: `http://host:port/log-out`. Funcionalidad: cerrar la sesión del usuario, eliminando su identificación del sistema. Requisitos: deben eliminarse las variables de sesión tanto del cliente como del servidor.

Tras una desconexión exitosa, el usuario debe ser redirigido a una página que informe que ha salido del sistema, y cinco segundos después, debe redirigirse automáticamente a la página de inicio.

- `songs` (véase figura 4). URL: `http://host:port/songs/:id`. Requisitos: esta página permite a los usuarios escribir las palabras omitidas en las letras de las canciones reproducidas. Dado que su funcionalidad es relativamente compleja, se le dedicará una sección específica.

- `faq` URL: `http://host:port/faq`. Requisitos: esta página reúne respuestas claras y concisas a las preguntas más comunes de los usuarios sobre un producto.

2.1 Página "Songs"

La página *songs* constituye el núcleo de la aplicación. En esta vista, el usuario podrá escuchar una canción mientras completa los espacios en blanco de la letra, los cuales han sido omitidos de forma intencionada.

Funcionalidades principales

- **Reproductor de audio:** se implementará un componente que proporcione un reproductor con controles de reproducción, pausa y navegación.

- **Visualización de la letra con huecos:** otro componente mostrará la letra de la canción con ciertas palabras omitidas, representadas por campos de texto (`<input>`) para que el usuario las complete.

- **Interacción del usuario:** el usuario escribe las palabras que escucha en los huecos. La validación se realizará en tiempo real. Si la palabra tecleada es correcta la aplicación seguirá reproduciendo la canción en caso contrario, esta se para hasta que se teclee la palabra correcta o se presione el botón "skip". Si no se teclea ninguna palabra ni se pulsa el botón "skip" la reprodución se parará al alcanzar el final de la frase.

- **Retroalimentación:** al terminar la canción se muestra el porcentaje de aciertos y los errores cometidos. Si el usuario está logeado esta información se almacena en el modelo `SongUser`. Se consideran fallos cada pulsación del botón "skip" así como cada palabra tecleada que no sea correcta. Si se realizan dos intentos para acerta una misma palabra se contabilizarán dos fallos.

- **Sincronización:** en cada instante sólo se mostrarán 3 líneas de la letra de la canción. La que se este cantando en ese momento y tanto la anterior como la siguiente. La línea actual de la canción debe resaltarse.

A grandes rasgos es necesario implementar una vista `PlayView.vue` que carge dos componentes: `AudioPlayer.vue` y `LyricsDisplay.vue`. A continaución se muestra el flujo de datos y eventos relacionados con `PlayView.vue`:

```
PlayView.vue
|
|-- realiza petición a API REST solicitando informacion sobre una canción
|
|-- pasa props a AudioPlayer.vue:
|     \-- song (objeto de tipo Song), stopAudio (boleano)
|
|-- pasa props a Lyricsdisplay.vue:
|     \-- song (objeto de tipo Song), onTimeUpdate (tiempo ejecución),
|
-- escucha eventos de:
|    |-- AudioPlayer.vue -> onTimeUpdate (tiempo ejecución)
|    \-- LyricsDisplay.vue -> stopAudio, startAudio, summary
\--actualiza SongUser si necesario (fetch al terminar la reproducción de la canción)
```

El componente principal `PlayView.vue` orquesta la interacción entre varios subcomponentes y la API. A continuación se describe el flujo de datos y eventos:

- **Carga de datos:** `PlayView.vue` realiza una petición `fetch` a una API para obtener:

  - `audioURL`
  - `lyricsURL`
  - `backgroun_image`
  - y quiza otros datos

- **Paso de props:**
  - A `AudioPlayer.vue` se le pasa los props:
    * `song` que contiene el atributo `audioURL`.
    * `stopAudio` si es igual a true la reproducción del audio debe detenerse
  - A `LyricsDisplay.vue` se le pasan los props:
    * `song` que contiene el atributo `lrc_file`
    * `onTimeUpdate`: (tiempo de ejecución).

- **Escucha de eventos:**
  - Desde `AudioPlayer.vue`:
    * `onTimeUpdate`: proviene de `Song_Player` y se reenvía a `LyricsDisplay`.
  - Desde `LyricsDisplay.vue`:
    * `summary`: contiene al número de respuestas correctas y el número de respuestas erróneas. Si el usuario está autentificado debe usarse esta información para crear una instancia de la clase `SongUser`.
    * `StartAudio` y `StopAudio`: informan al reproductor si puede contiar o debe parar la ejecución del audio.

- **Actualización del usuario:** Si es necesario, `play.vue` realiza un `fetch` para actualizar el estado del usuario (`SongUser`) con los resultados obtenidos.

Formato de las letras sincronizadas

Las letras de las canciones se almacenarán en un formato de texto plano que incluye metadatos y marcas de tiempo para sincronizar cada línea con el audio. Este formato esta basado en el utilizado por los archivos `.lrc`.

A continuación se muestra un ejemplo del formato:

```
[ti:Does Your Mother Know]
[ar:ABBA]
[lang:English]
[length:03:13.00]
[by:Jun]
[re:rclyricsband.com]
[ve:v0.0.5]
[00:00.00]
[00:22.48]You're so {hot}, teasing me
[00:25.81]So you're {blue} but I can't take a chance on a chick like you
[00:32.10]That's something I couldn't do
```

Descripción del formato:

- Las primeras líneas contienen metadatos y no los usaremos en esta aplicación
- Las líneas con marcas de tiempo `[mm:ss.xx]` indican el momento exacto en que debe mostrarse la línea correspondiente de la letra. El entero `mm` se refiere a los minutos y numero real `ss.xx` a los segundos
- Las palabras que deben ser omitidas para que el usuario las complete se marcan rodeándolas con llaves {...}. Estas palabras se reemplazarán por campos de entrada en la interfaz de usuario.
- Nuestra aplicación no soporta el uso de varias voces de forma simultánea
- Nuestra aplicación no soporta la omisión de dos palabras en la misma línea
- Si la canción tiene un estribillo el mismo deverá incluirse varias veces en el fichero con la letra, Esto es no se soporta entradas como:

  ```
  [00:58.83][01:01.12]Flirt a little maybe
  ```

Cómo implementar un componente AudioPlayer.vue

Podéis usar el elemento de HTML5 `<audio>`

Listado 2: listado parcial que muestra como usar el elemento `<audio>` y los eventos `timeUpdate` y `ended`

```
// AudioPlayer.vue

<audio
    ref="audio"
    :src="audioSrc"
    @timeupdate="emitTimeUpdate"
    @ended="emitEnded"
></audio>
...
<script setup>
import { ref } from 'vue'
const audio = ref(null)

function emitTimeUpdate() {
  emit('onTimeUpdate', audio.value.currentTime)
}
...
function emitEnded() {
  emit('onEnded')
}
...
</script>
```

El elemento `audio` emite varias veces por segundo el evento `emitTimeUpdate`. Es posible asociarle al evento una función "callback", en este ejemplo `emitTimeUpdate`. Esta función recoge el tiempo de ejecución (`currentTime`), y emite el evento `onTimeUpdate` que debe ser recogido por la aplicación padre y pasado al componente hijo `Lyrics.vue`

3 Despliegue de la aplicación

Tanto el frontend como el backend de la aplicación deben desplegarse en `render.com`. Los URL de las dos proyectos deben figurar en un fichero llamado `.env` situado en la raíz del proyecto. Igualmente en este fichero se debe encontrar la URI de la base de datos creada en `neon.tech` y asada por la aplicación.

4 Devtools

Tal y como se describió en la práctica 2 activad `devtools` en la versión de producción desplegada en *render.com*.

5 Test

En el repositorio accesible en *Moodle* podéis encontrar el test `frontend_test`. Este test asume que: (a) la aplicación está poblada con la canción "Here in the real world" (los ficheros pertinente se encuentran disponibles en el repositorio concretamente en el directorio `Media`) y (b) existe un usuario llamado `alumnodb` con password `alumnodb`. El test realiza las siguientes acciones:

- Conectarse a la página principal
- comprobar que existe el botón "random song"
- logearse en el sistema
- buscar una canción
- reproducir esta canción y rellenar los huecos en la letra
- comprobar que se reporta el numero de respuestas acertadas y falladas correctamente (recordad que cada intento cuenta como un fallo aunque se realicen varios intentos en la misma palabra)

A la terminación del test habría que comprobar que se ha creado una nueva entrada en la tabla `songuser` con los valores pertinentes y que el atributo `song.number_times_played` se ha incrementado en uno.

6 Resumen del trabajo a entregar

- Aseguraros que vuestro código satisface el test proporcionado. No es admisible que se modifique el código del mismo.
- Desplegad y probad la aplicación en `render.com` en modo de producción
- Subid a *Moodle* el fichero obtenido al ejecutar el comando `zip -r ../assign.zip .git` desde la raíz del proyecto (este fichero debe contener tanto el backend como el frontend). Recordad que hay que añadir y enviar (commit) los ficheros a git antes de ejecutar el comando. Si queréis comprobar que el contenido del fichero zip es correcto lo podéis hacer ejecutando la orden: `cd ..; unzip assign.zip; git clone . tmpDir; ls tmpDir`.

  IMPORTANTE: nos hará falta el URI para acceder a la base de datos creada en `neon.tech` y el URL en el que se ha desplegado la aplicación en `render.com` para evaluar esta práctica. Por favor, incluidlo en un fichero llamado `.env` que se encuentre en la raíz del proyecto. Si no se encuentra esta información la práctica estará suspendida. Asimismo, comprobar en *render.com* que existe un usuario administrativo con username y password `alumnodb`.

7 Criterios de evaluación

Para obtener la clasificación de apto se deben satisfacer todos los criterios siguientes:

- La aplicación está desplegada en *render.com* usando el mismo codigo que se ha subido a *Moodle*.

- Las URLs en *render.com* deben seguir el formato: `http://practica\_proyecto\_pareja\_grupo\_agno\_version.onrender.com` (en este caso: práctica = 4, proyecto = *songproject*, etc. `version` es una cadena arbitraria que el alumno puede elegir libremente.
- El test proporcionado (sin modificaciones) se puede ejecutar apuntando a *render.com* y se ejecuta sin reportar ningún fallo
- Al finalizar el test las tablas `songuser` y `song` se han actualizado adecuadamente.
- La funcionalidad solicitada en la página `songs` se ha implementado usando vistas y componentes que se comunican mediante `props` y `emits` tal y como se describe en la documentación de la práctica.
- Existe un fichero `.env` en la raíz del proyecto con la información solicitada.
- Devtools está accesible en la aplicación desplegada en *render.com*.

Nota: Entrega fuera de plazo →, se substraerá un punto de la nota del examen por cada día (o fracción) de retraso en la entrega.

Nota: El código usado en la corrección de la práctica será el entregado en *Moodle*. Bajo ningún concepto se usará el código existente en *render.com*, *Github* o cualquier otro repositorio.

A Capturas de pantalla

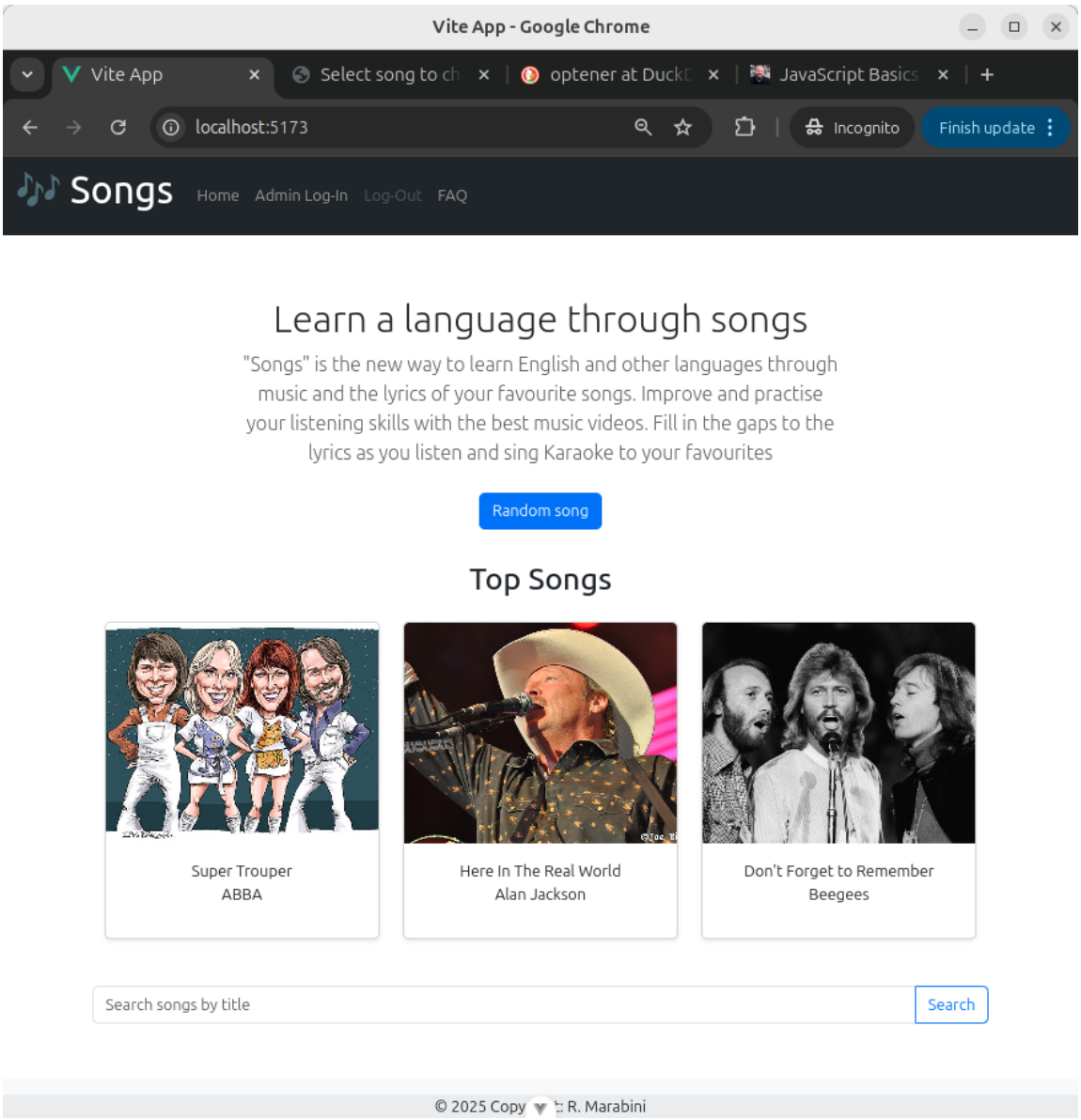

Figure 1: Pagina inicial de la aplicación para usuarios no logeados mostrando un listado con las canciones más populares, la posibilidad de realizar búsquedas y un botón para acceder a una canción al azar.

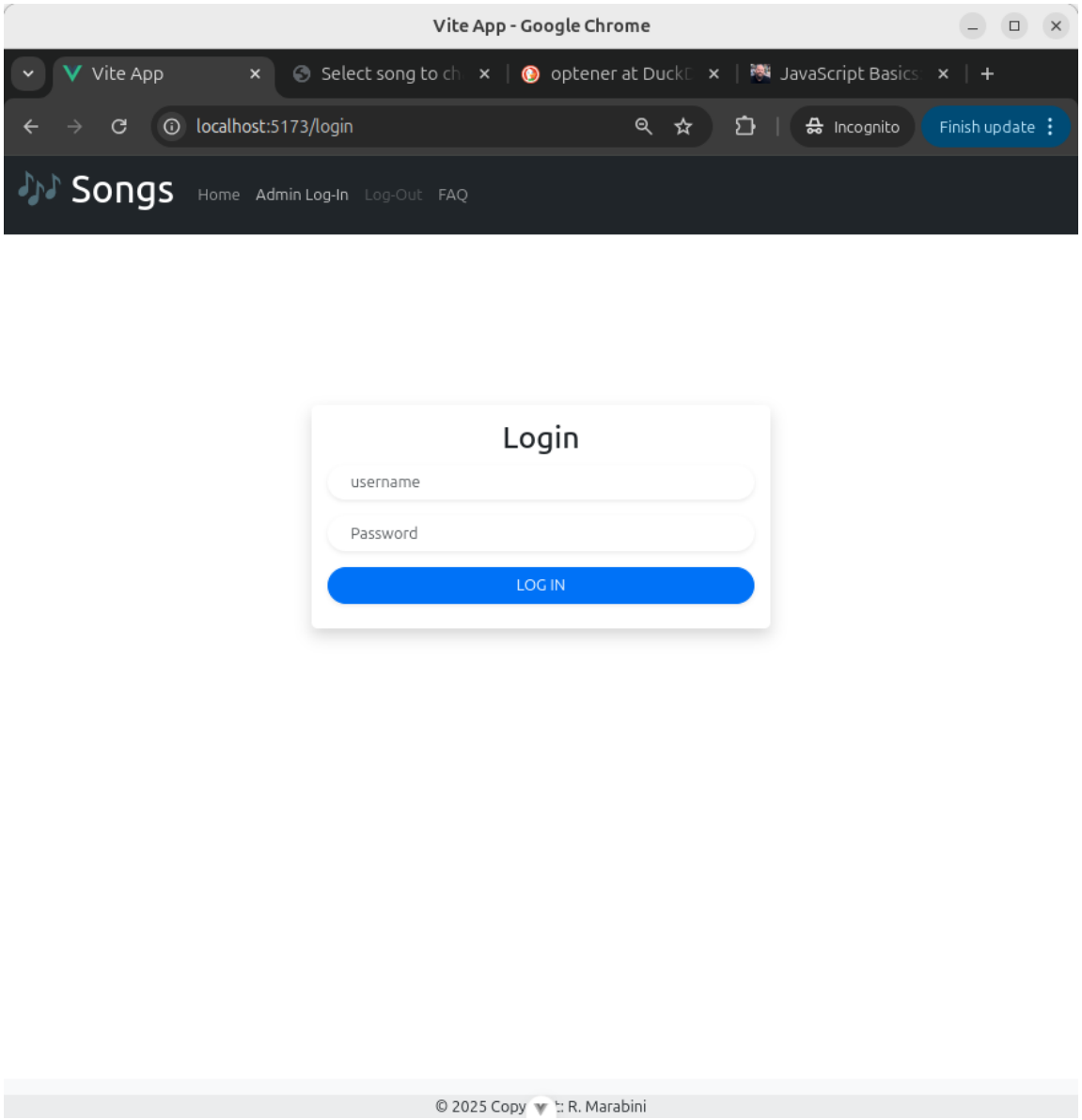

Figure 2: "login page" (o página de autenticación) en la que los usuarios pueden autenticarse.

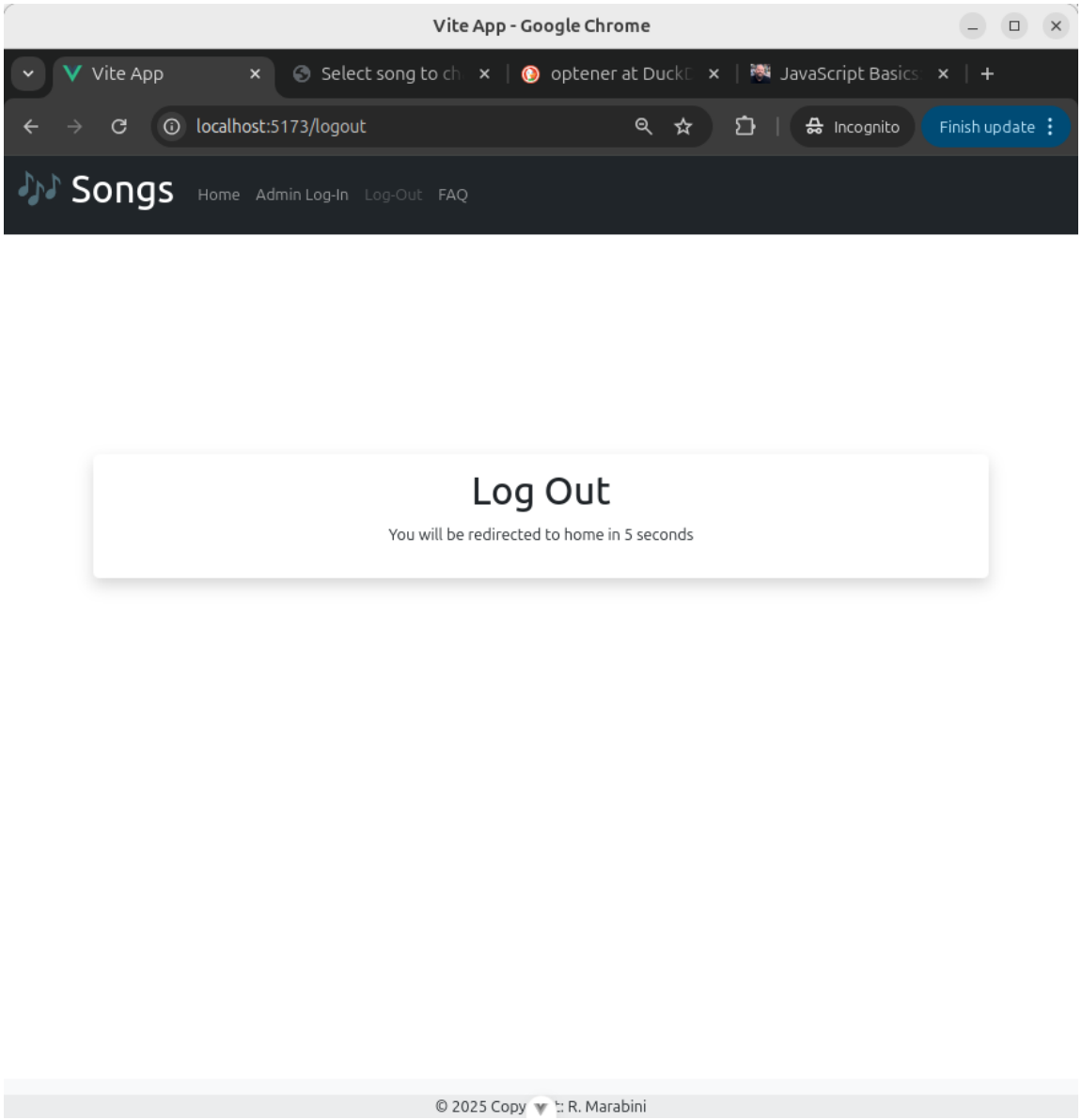

Figure 3: "Log-Out page" (o página de cierre de sesión) que permite a los usuarios cerrar la sesión actual.

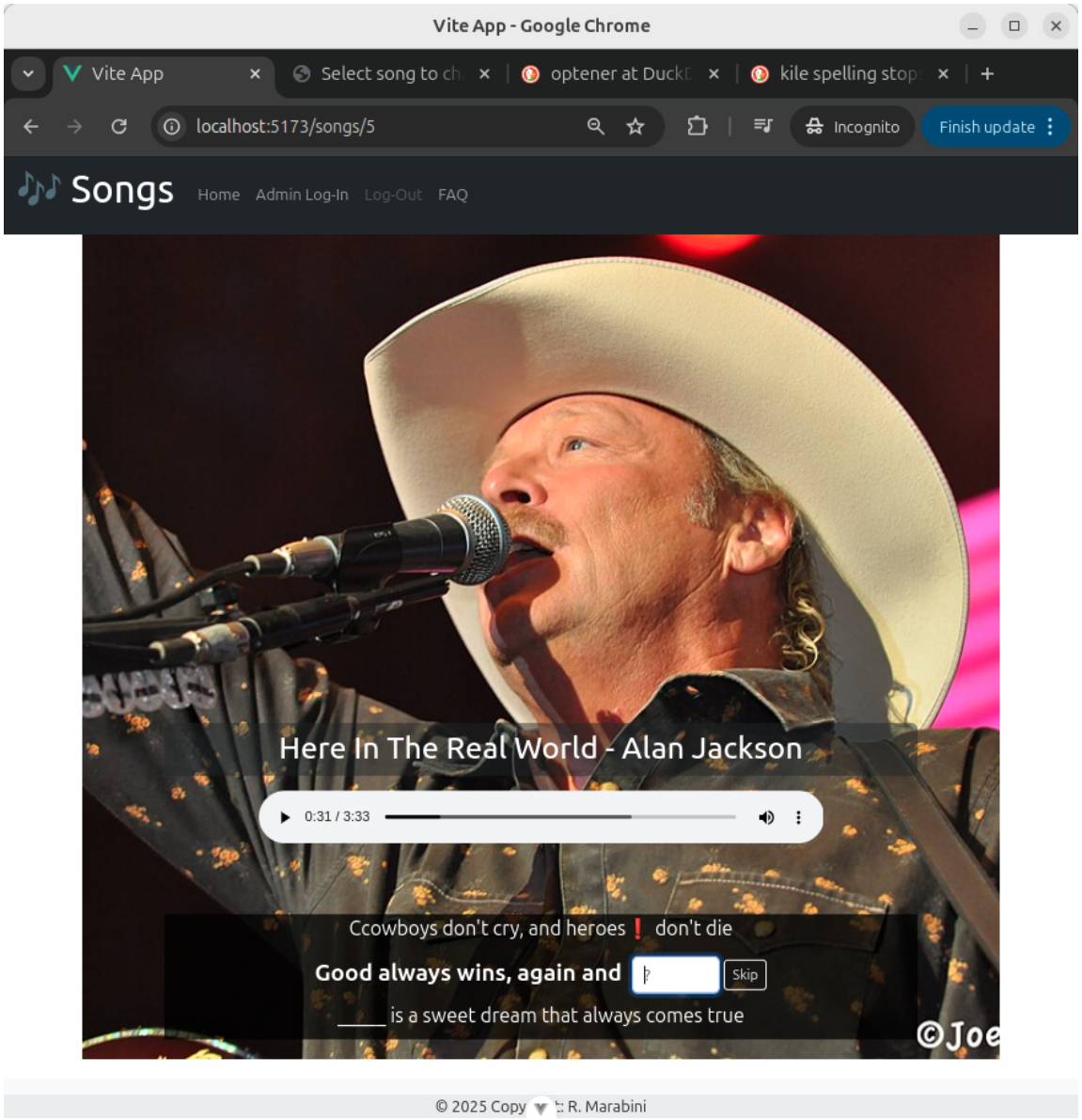

Figure 4: Página donde los usuarios pueden escuchar una canción al tiempo que pueden escribir las palabras que faltan en la letra.